UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

Facultad de Ciencias Químicas e Ingeniería

Práctica 3

Materia:
Organización y Arquitectura de Computadoras

Docente:
Garcia Rocha Jose Isabel

Elaborado por:
Liang Liu Kevin

Tijuana, B.C. a 04 de marzo de 2026

Objetivo

Seleccionar el modo de direccionamiento adecuado, para manejo de memoria en aplicaciones de sistemas basados en microprocesador, mediante el análisis de los diversos modos de direccionamiento existentes, de forma responsable y eficiente.

Desarrollo

1. Capturar dos números y guardarlos en sus respectivas variables en memoria no inicializada.

```
C/C++
section .data
  msj1: db 'Ingrese un numero (1-9)',0x0
  len1: equ $-msj1

section .bss
    num1: resb 2
    num2: resb 2
    cadena: resb 2

section .text
  _start:

;COPIA msj1 A edx Y LO MUESTRA
  mov edx, msj1
  call puts
  call salto

;PIDE UN CARACTER Y LO GUARDA A num1
  call getch
  mov ebx, num1
  mov [ebx], al
  call putchar
  call salto

;MUESTRA msj1 EN edx
  call puts
  call salto

;PIDE UN CARACTER Y LO GUARDA A num2
  call getch
  mov ebx, num2
  mov [ebx], al
  call putchar
```

```
  call salto

;CONVIERTE num1 DE CARACTER A NUMERO
   mov ebx, num1
   sub byte[ebx], '0'

;CONVIERTE num2 DE CARACTER A NUMERO
   mov ebx, num2
   sub byte[ebx], '0'
```

```
● @liang8451 →/workspaces/OAC_2026_01/Practica3 (main) $ ./codigo
Ingrese un numero (1-9)
4
Ingrese un numero (1-9)
3
```

2. Implementar un procedimiento que por medio de sumas consecutivas utilizando la instrucción loop explicada en clase genere el mismo funcionamiento que una multiplicación.

a. Ejemplo: 3x4=12
   i. 4+4+4=12.
   ii. El primer número indica el número de veces a sumar el segundo número.

```
C/C++
;MULTIPLICACION
  mov ebx, num1
  mov ecx, [ebx] ;CONTADOR DEL LOOP, EMPIEZA CON num1
  mov al, 0 ;al GUARDA LA SUMA
  mov ebx, num2

  sumar:
    add al, byte[ebx] ;SUMA num2 A al
  loop sumar

  mov esi, cadena
  call printHex
  call salto
```

```
● @liang8451 →/workspaces/OAC_2026_01/Practica3 (main) $ ./codigo
Ingrese un numero (1-9)
9
Ingrese un numero (1-9)
9
9x9=00000051
```

Nota: 51h = 81

3. Implementar un procedimiento que por medio de restas consecutivas utilizando la instrucción jmp explicada en clase genere el mismo funcionamiento que una división de enteros.

   a. Ejemplo: 12/3=4
      i. 12-3=9; 9-3=6; 6-3=3; 3-3=0.
      ii. El primer número es el resultado de la anterior función, el segundo número es el que se le restara al primero hasta dar con el resultado de 0.

```C/C++
;DIVISION
  mov cl, 0 ;cl GUARDA LAS VECES QUE SE RESTA
  mov ebx, num1
  mov eax, [ebx] ;eax GUARDA num1
  mov ebx, num2
  mov ebx, [ebx] ;ebx GUARDA num2

  cmp ebx, 0    ;DIVISION ENTRE 0
  je salirRestar

  restar:
  cmp eax, 0
  je salirRestar
  sub eax, ebx
  inc cl
  jmp restar

  salirRestar:
  mov al, cl
  mov esi, cadena
  call printHex
```

```
● @liang8451 →/workspaces/OAC_2026_01/Practica3 (main) $ ./codigo
Ingrese un numero (1-9)
9
Ingrese un numero (1-9)
3
9/3=00000003
```

4. Implementar un procedimiento que muestre los números desde el 0 al 100 utilizando la subrutina printHex que se les entregó en moodle, únicamente utilizando las instrucciones vistas hasta el momento.

a. Ejemplo: 0, 1, 2, 3, … , 100

```
C/C++
  mov ecx, 101 ;VECES A REPETIR EL LOOP
  mov bl, 0 ;CONTADOR EMPIEZA EN 0
  mov esi, cadena

  contador:
    mov al, bl
    call printHex
    mov al, ' '
    call putchar
    inc bl
    loop contador

  call salto
```

```
● @liang8451 →/workspaces/OAC_2026_01/Practica3 (main) $ ./codigo
00000000 00000001 00000002 00000003 00000004 00000005 00000006 00000007 00000008 00000009
0000000A 0000000B 0000000C 0000000D 0000000E 0000000F 00000010 00000011 00000012 00000013
00000014 00000015 00000016 00000017 00000018 00000019 0000001A 0000001B 0000001C 0000001D
0000001E 0000001F 00000020 00000021 00000022 00000023 00000024 00000025 00000026 00000027
00000028 00000029 0000002A 0000002B 0000002C 0000002D 0000002E 0000002F 00000030 00000031
00000032 00000033 00000034 00000035 00000036 00000037 00000038 00000039 0000003A 0000003B
0000003C 0000003D 0000003E 0000003F 00000040 00000041 00000042 00000043 00000044 00000045
00000046 00000047 00000048 00000049 0000004A 0000004B 0000004C 0000004D 0000004E 0000004F
00000050 00000051 00000052 00000053 00000054 00000055 00000056 00000057 00000058 00000059
0000005A 0000005B 0000005C 0000005D 0000005E 0000005F 00000060 00000061 00000062 00000063
00000064
```

5. Implementar un procedimiento que muestre los números pares desde el 0 al 100, utilizando la instrucción cmp y el salto condicional je.

```
C/C++
   mov bl, 0 ;CONTADOR EMPIEZA EN ZERO
```

```
contadorPar:
   mov al, bl
   call printHex
   mov al, ' '
   call putchar
   cmp bl, 100 ;LIMITE SUPERIOR DEL CONTADOR
   je salirContadorPar
   inc bl
   inc bl
   jmp contadorPar

salirContadorPar:
```

```
@liang8451 →/workspaces/OAC_2026_01/Practica3 (main) $ ./codigo
00000000 00000002 00000004 00000006 00000008 0000000A 0000000C 0000000E 00000010 00000012
00000014 00000016 00000018 0000001A 0000001C 0000001E 00000020 00000022 00000024 00000026
00000028 0000002A 0000002C 0000002E 00000030 00000032 00000034 00000036 00000038 0000003A
0000003C 0000003E 00000040 00000042 00000044 00000046 00000048 0000004A 0000004C 0000004E
00000050 00000052 00000054 00000056 00000058 0000005A 0000005C 0000005E 00000060 00000062
00000064
```

Dificultades

Mientras se realizaba el programa de multiplicación, se encontró con el problema de imprimir en consola un número mayor de 9, porque en ese caso se tiene que imprimir >2 dígitos. Al final se uso la función *printHex* para imprimir el resultados.

El programa de división está limitado a recibir números de 0 a 9, porque se usa la subrutina *getch* que solo permite un carácter. También solo con un par de números que sean divisibles.

Conclusiones

Se logró con éxito el objetivo de analizar y aplicar diversos modos de direccionamiento, así como instrucciones clave del lenguaje ensamblador x86 (como mov, add, sub, cmp, loop, y jmp), para manipular la memoria y simular operaciones aritméticas complejas.

Los ejercicio de iteración y uso de saltos condicionales reforzó la comprensión de cómo se construyen estructuras de control básicas (bucles for y while) en un entorno de programación de bajo nivel.